AI Notes

Key Prompts — Log Anomaly Explainer Project

■ Backend — Log Parser

Prompt 1 — log_parser.py

Create log_parser.py with a function find_error_block(log_path: str, context_lines: int = 20) -> dict. Requirements: Read the entire .log file. Detect error blocks using regex patterns for common errors: ERROR, CRITICAL, Exception, Traceback, stack trace, etc. Return the main error block + 20 lines before/after as context. Also return timestamp if possible, severity, and raw snippet. Handle large files efficiently. Add tests with a sample.log in examples/. Use clean, documented code.

■ LLM Explainer

Prompt 2 — llm_explainer.py (v1)

Create llm_explainer.py that uses Ollama to explain log errors. Function: explain_anomaly(error_context: str, model: str = 'llama3.1') -> str. Prompt template should ask for: probable root cause, why it happened, suggested fix, prevention tips. Keep response structured (use markdown sections). Handle Ollama client properly with error fallback. Make it fast (temperature 0.3, max_tokens reasonable).

Prompt 3 — llm_explainer.py (v2, structured output)

Create llm_explainer.py with proper Ollama integration. Function: explain_anomaly(log_context: dict, model: str = 'llama3.1:8b') -> dict. Input is the dict returned by find_error_block(). Use ollama python library. Craft a strong system + user prompt that asks the LLM to: (1) Summarize what went wrong in 1-2 sentences, (2) Probable root cause, (3) Why it likely happened, (4) Suggested immediate fix, (5) Prevention tips for future. Return structured dict: {summary, root_cause, suggested_fix, prevention, raw_llm_response}. Add timeout / error handling (if Ollama not running -> friendly error). Temperature=0.3. Add 2-3 example calls in __main__ for testing. Clean, well-documented code with type hints.

Prompt 4 — Fix Section Parsing

Fix the section parsing in llm_explainer.py so that structured fields are properly populated. Current problem: explain_anomaly() returns empty sections even though raw_llm_response contains good content. Make _parse_sections() more robust using regex or string matching for: SUMMARY, ROOT_CAUSE, WHY_IT_HAPPENED, SUGGESTED_FIX, PREVENTION. Handle variations in LLM output (bold, uppercase, different headings). Fallback: if parsing fails, put the entire raw response into 'summary' field. Test it with sample.log and ensure the final report shows proper content in the AI Analysis section.

■■ CLI — main.py

Prompt 5 — main.py CLI (v1)

Create main.py CLI using typer. Command: log-anomaly [--model llama3.1] [--output report.md]. Use log_parser to get error block + context. Use llm_explainer. Generate nice Markdown report with: Summary, Error Snippet, LLM Explanation, Suggested Fix, Timestamp. Use rich for nice terminal output. Add --help and good UX.

Prompt 6 — main.py CLI (v2, full options)

Create main.py as the CLI entry point using typer. Options: --model TEXT (default: 'llama3.2:latest'), --output TEXT (default: 'anomaly_report.md'), --context-lines INT (default: 20), --no-llm (flag: parse only, no

explanation). Workflow: (1) Call find_error_block(logfile, context_lines), (2) If found, call explain_anomaly(result, model), (3) Generate a beautiful Markdown report with sections: # Log Anomaly Report, File, Timestamp, Severity, Error Summary, Raw Error Block.

■ Polish & Production-Ready

Prompt 7 — Full Project Polish

Polish the entire log-anomaly-explainer project to make it production-ready. Tasks: (1) Fix any issues in main.py + report_generator.py so that full LLM explanation appears in the Markdown report. (2) Improve the report Markdown: more visually appealing, properly integrate all fields from explain_anomaly() output, add a clear 'AI Analysis' section with proper headings, include severity badge or emoji. (3) Update README.md with: project description, installation steps (Ollama + pip), usage examples. (4) Update requirements.txt with all dependencies. (5) Add a simple .gitignore. (6) Optional: Add --auto-open flag to open the report in browser after generation.

■ Streamlit Web App

Prompt 8 — app.py (v1, basic)

Create a complete single-file Streamlit app in app.py for the Log Anomaly Explainer project. Requirements: Professional dark theme with nice animations. Use tabs: 'Analyze Log', 'Dashboard', 'History'. Integrate with existing backend (python -m backend.main). Use SQLite to save analysis history. Support file upload, analyze button, show full markdown report. Download Markdown report. Include custom CSS for modern look, error handling, auto-create uploads/ and reports/ folders. Store in DB: filename, report_path, created_at, summary.

Prompt 9 — app.py (v2, improved features)

Improve app.py with: Better report parsing to extract Summary, Root Cause, Suggested Fix, Prevention. Add PDF download using markdown2 and weasyprint (if possible). Dashboard showing total analyses and recent activity. History page with search and expandable reports. Make the Analyze tab the default and most prominent. Add loading spinners and success messages.

Prompt 10 — app.py (v3, premium polish)

Final polish for app.py: Make it look premium (gradients, cards, emojis). Add sidebar with quick links. Handle backend errors gracefully. Auto-refresh history. Add a 'Clear History' button. Update README.md with Streamlit run instructions.

Prompt 11 — app.py (v4, full backend integration)

Update the current Streamlit app (app.py) to fully integrate with the existing backend and database. Key Requirements: In 'Analyze Log' tab: When user uploads file and clicks Analyze, call the existing backend CLI. Save full analysis to SQLite database (filename, severity, summary, report_path, etc.). Display the full generated Markdown report nicely after analysis. Add Download buttons for both .md and .pdf (if possible). Make 'Dashboard' tab show total analyses, recent activity. 'History' tab should list past analyses with search and expandable report preview. Keep the attractive dark theme and animations. Handle errors gracefully.

Prompt 12 — app.py (v5, final fix)

Fix the current Streamlit app (app.py) completely. The app must: (1) Have dark professional theme. (2) Tabs: Analyze Log (main), Dashboard, History. (3) In Analyze Log: File uploader for .log files, Analyze button that calls python -m backend.main, Display full markdown report after success, Save to SQLite database, Download Markdown button. (4) Dashboard: Show total analyses count and simple stats. (5) History: Show past analyses with search and expandable details. Use try-except blocks everywhere to avoid JSON or runtime errors. Make sure folders (uploads, reports) are created automatically. Keep it simple but attractive.